



Understanding Indexers in C#.NET: A Comprehensive Guide

By Albert Harmon Published October 23, 2016 ⌚ 4 min read [Education](#)



TLDR This blog post explains the concept of indexers in C#.NET, demonstrating how they allow classes to behave like virtual arrays. It covers the definition of indexers, their implementation in a class, and how to access and modify class attributes using

indexers, along with examples and best practices.

In this tutorial, we will explore the concept of indexers in C#.NET. An indexer is a member of a class that allows instances of that class to be indexed just like arrays. This feature enables a class to behave like a virtual array, providing a convenient way to access its values.

What are Indexers?

Indexers are special properties that allow you to access elements in a class using an index. They provide a way to access class attributes in a manner similar to accessing elements in an array. For example, if you have a class representing an employee, you can access the employee's attributes using index positions.

Defining a Class with Attributes

To illustrate the use of indexers, let's define a class called ``Employee`` that contains several attributes:

```
public class Employee
{
    private int eNumber;
    private string eName;
    private string job;
```

```
private double salary;
private string departmentName;
private string location;

public Employee(int eNumber, string eName, string job, dou
{
    this.eNumber = eNumber;
    this.eName = eName;
    this.job = job;
    this.salary = salary;
    this.departmentName = departmentName;
    this.location = location;
}
}
```

In this class, we have defined six attributes: employee number, name, job, salary, department name, and location. These attributes are private by default, meaning they cannot be accessed directly from outside the class.



Accessing Class Attributes

To access the attributes of the ``Employee`` class, we can create an instance of the class in another class, such as ``TestEmployee`` :

```
public class TestEmployee
{
    static void Main(string[] args)
    {
        Employee emp = new Employee(101, "Scott", "Manager", 2
    }
}
```

However, since the attributes are private, we cannot access them directly using `emp.eNumber`` , for example. To provide access to these attributes, we have several options:

1. **Make the attributes public:** This is not recommended as it exposes the internal state of the class.
2. **Use properties:** Properties can provide controlled access to the attributes.
3. **Use indexers:** This allows access to the attributes using index positions.

Implementing Indexers

To implement an indexer in the `Employee`` class, we define it as follows:

```
public object this[int index]
{
    get
    {
        switch (index)
        {
            case 0: return eNumber;
            case 1: return eName;
            case 2: return job;
            case 3: return salary;
            case 4: return departmentName;
            case 5: return location;
            default: return null;
        }
    }
}
```

This indexer allows us to access the employee's attributes using an index. For example, `emp[0]` will return the employee number, `emp[1]` will return the employee name, and so on.

Accessing Values Using Indexers

Once the indexer is defined, we can access the values in the `TestEmployee` class:

```
Console.WriteLine(emp[0]); // Outputs: 101
Console.WriteLine(emp[1]); // Outputs: Scott
```

Modifying Values Using Indexers

If we want to allow modification of the values, we need to implement a `set` accessor in the indexer:

```
public object this[int index]
{
    get { /* existing get logic */ }
    set
    {
        switch (index)
        {
            case 0: eNumber = (int)value; break;
            case 1: eName = (string)value; break;
            case 2: job = (string)value; break;
            case 3: salary = (double)value; break;
            case 4: departmentName = (string)value; break;
            case 5: location = (string)value; break;
        }
    }
}
```

Now, we can modify the employee's attributes using the indexer:

```
emp[2] = "Senior Manager"; // Changes job to Senior Manager  
emp[3] = 25000; // Changes salary to 25000
```

Conclusion

In this tutorial, we have covered the concept of indexers in C#.NET, demonstrating how they allow classes to behave like virtual arrays. By defining an indexer, we can provide controlled access to class attributes using index positions. This feature enhances the usability of classes and encapsulates the internal state effectively.

For further learning, consider exploring properties and other advanced features in C#.NET to enhance your programming skills.